

Pemrograman Mobile Flutter

Minggu 5 — Local Storage & Offline First

Project: `week5_offline_notes` • Stack: Riverpod, SharedPreferences, sqflite, Dio

Nama	Rakagali Resda Krisandi Putra
NIM	244107020136
Kelas	TI-3C
Link repository	https://github.com/Rageel-28/244107020136-mobile-course/tree/main/05-week-5-local-storage-offline-first

Cara memakai laporan ini

1. Kerjakan **berurutan** dari Praktikum 1 sampai 5. Tiap praktikum memakai hasil sebelumnya.
2. Setiap blok kode punya label file di bagian atas dan tanda **[BUAT FILE BARU]** atau **[GANTI SELURUH ISI]**. Ikuti persis, tinggal ketik/salin ke file dengan nama yang sama.
3. Kotak putus-putus = tempat tempel **screenshot**. Nama file yang harus dipakai tertulis di bawah kotak.
4. Kolom **Hasil aktual** dan **Status** pada tabel uji diisi sendiri setelah kamu mencoba.

Peta kerja

Bagian	File yang dibuat / diubah	Screenshot	Commit
Setup	<code>pubspec.yaml</code> , folder project	p0-pubspec, p0-counter	chore: init project week5_offline_notes
Praktikum 1	<code>prefs.dart</code> , <code>prefs_providers.dart</code> , <code>settings_page.dart</code> , <code>main.dart</code>	p1-tema-terang, p1-tema-gelap	feat(prefs): simpan tema dan waktu terakhir dibuka
Praktikum 2	<code>note.dart</code> , <code>db.dart</code> , <code>note_repository.dart</code>	p2-analyze	feat(db): model Note, skema SQLite, dan NoteRepository
Praktikum 3	<code>note_providers.dart</code> , <code>note_form_dialog.dart</code> , <code>notes_page.dart</code> , <code>main.dart</code>	p3-empty, p3-tambah, p3-validasi, p3-mode-pesawat	feat(notes): halaman catatan offline dengan Riverpod
Praktikum 4	<code>offline_providers.dart</code> , <code>sync.dart</code> , <code>post.dart</code> , <code>post_repository.dart</code> , <code>post_providers.dart</code> , <code>posts_page.dart</code> + ganti 3 file lama	p4-dirty-sebelum, p4-dirty-sesudah, p4-posts-offline	feat(sync): cache-first posts, dirty flag, dan sinkronisasi
Praktikum 5	<code>test/note_test.dart</code>	p5-flutter-test	test: model, provider, dan sync dengan repository palsu
AI Challenge	<code>docs/perbandingan-storage.md</code> , <code>docs/uji-offline.md</code>	—	docs: perbandingan storage dan hasil uji offline

Setup Project

Sekali saja di awal. Jalankan di emulator Android / HP asli, bukan di Chrome (sqlite tidak mendukung web).

Langkah 1 Cek Flutter

Terminal [TERMINAL]

```
flutter --version  
flutter doctor
```

Pastikan bagian **Android toolchain** tidak ada tanda **X** dan ada perangkat/emulator yang terdeteksi.

Langkah 2 Buat project dan pasang paket

Terminal [TERMINAL]

```
flutter create week5_offline_notes  
cd week5_offline_notes  
flutter pub add flutter_riverpod shared_preferences sqlite path dio
```

Langkah 3 Buat folder

Pilih salah satu sesuai terminalmu:

Git Bash / macOS / Linux [TERMINAL]

```
mkdir -p lib/data/local lib/data/remote lib/data/repositories  
mkdir -p lib/providers lib/pages lib/widgets  
mkdir -p docs screenshots
```

PowerShell (Windows) [TERMINAL]

```
New-Item -ItemType Directory -Force lib/data/local, lib/data/remote, lib/data/repositories, lib/providers,  
lib/pages, lib/widgets, docs, screenshots
```

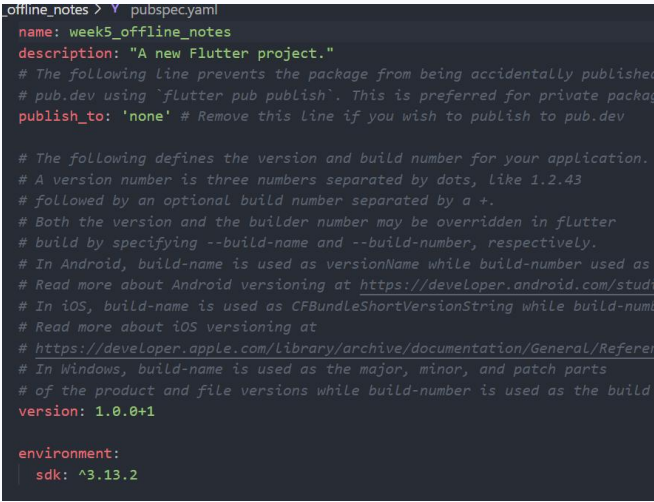
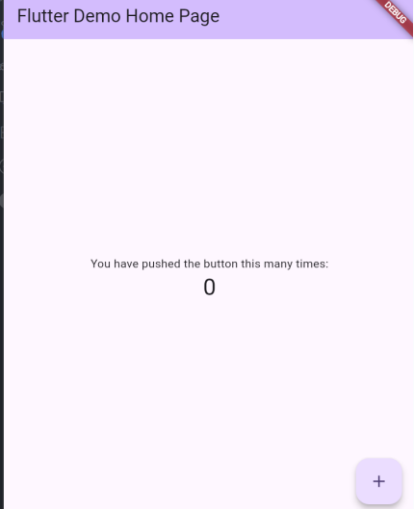
Langkah 4 Jalankan aplikasi bawaan

Terminal [TERMINAL]

```
flutter run
```

Harus terjadi

Aplikasi counter bawaan tampil di emulator, dan `pubspec.yaml` memuat 5 paket di bagian `dependencies`.

 <pre>offline_notes > Y pubspec.yaml name: week5_offline_notes description: "A new Flutter project." # The following line prevents the package from being accidentally published to # pub.dev using `flutter pub publish`. This is preferred for private packages publish_to: 'none' # Remove this line if you wish to publish to pub.dev # The following defines the version and build number for your application. # A version number is three numbers separated by dots, like 1.2.43 # followed by an optional build number separated by a +. # Both the version and the builder number may be overridden in flutter # build by specifying --build-name and --build-number, respectively. # In Android, build-name is used as versionName while build-number used as # build by specifying --build-name and --build-number, respectively. # Read more about Android versioning at https://developer.android.com/studio # In iOS, build-name is used as CFBundleShortVersionString while build-number # Read more about iOS versioning at # https://developer.apple.com/library/archive/documentation/General/Referen # In Windows, build-name is used as the major, minor, and patch parts # of the product and file versions while build-number is used as the build version: 1.0.0+1 environment: sdk: ^3.13.2</pre>	 Flutter Demo Home Page You have pushed the button this many times: 0 +
Gambar 1. Bagian dependencies di pubspec.yaml simpan: screenshots/p0-pubspec.png	Gambar 2. Aplikasi counter bawaan berjalan simpan: screenshots/p0-counter.png

Commit praktikum ini:

Terminal [TERMINAL]
<pre>git add . git commit -m "chore: init project week5_offline_notes"</pre>

Praktikum 1 — SharedPreferences

Target: tema gelap/terang + waktu terakhir dibuka tetap tersimpan setelah aplikasi ditutup.

Langkah 1 Repository preferensi

lib/data/prefs.dart [BUAT FILE BARU]

```
import 'package:shared_preferences/shared_preferences.dart';

class PrefsRepository {
  static const _darkModeKey = 'dark_mode';
  static const _lastOpenedKey = 'last_opened_at';

  Future<bool> getDarkMode() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getBool(_darkModeKey) ?? false;
  }

  Future<void> setDarkMode(bool value) async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setBool(_darkModeKey, value);
  }

  Future<void> markOpenedNow() async {
    final prefs = await SharedPreferences.getInstance();
    await prefs.setString(_lastOpenedKey, DateTime.now().toIso8601String());
  }

  Future<String?> getLastOpened() async {
    final prefs = await SharedPreferences.getInstance();
    return prefs.getString(_lastOpenedKey);
  }
}
```

Langkah 2 Provider preferensi

lib/providers/prefs_providers.dart [BUAT FILE BARU]

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/prefs.dart';

final prefsRepositoryProvider =
  Provider<PrefsRepository>((ref) => PrefsRepository());

final darkModeProvider =
  AsyncNotifierProvider<DarkModeNotifier, bool>(DarkModeNotifier.new);

class DarkModeNotifier extends AsyncNotifier<bool> {
  @override
  Future<bool> build() => ref.watch(prefsRepositoryProvider).getDarkMode();

  Future<void> toggle() async {
    final previous = state.value ?? false;
    final next = !previous;
    // Optimistic update: UI langsung berubah tanpa menunggu disk.
    state = AsyncData(next);
    try {
      await ref.read(prefsRepositoryProvider).setDarkMode(next);
    } catch (_) {
      // Gagal menyimpan: kembalikan ke nilai sebelumnya.
      state = AsyncData(previous);
      rethrow;
    }
  }
}
```

```

/// Membaca waktu terakhir dibuka (dari sesi sebelumnya),
/// lalu mencatat waktu sekarang untuk sesi berikutnya.
final lastOpenedProvider = FutureProvider<String?>((ref) async {
  final repo = ref.watch(prefsRepositoryProvider);
  final previous = await repo.getLastOpened();
  await repo.markOpenedNow();
  return previous;
});

```

Langkah 3 Halaman pengaturan

```

lib/pages/settings_page.dart [BUAT FILE BARU]

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../providers/prefs_providers.dart';

class SettingsPage extends ConsumerWidget {
  const SettingsPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final darkAsync = ref.watch(darkModeProvider);
    final lastOpenedAsync = ref.watch(lastOpenedProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Pengaturan')),
      body: ListView(
        children: [
          darkAsync.when(
            loading: () => const ListTile(
              title: Text('Memuat preferensi...'),
              trailing: SizedBox(
                width: 24,
                height: 24,
                child: CircularProgressIndicator(strokeWidth: 2),
              ),
            ),
            error: (e, _) => ListTile(
              leading: const Icon(Icons.error_outline),
              title: const Text('Gagal memuat preferensi'),
              subtitle: Text('$e'),
            ),
            data: (isDark) => SwitchListTile(
              secondary: const Icon(Icons.dark_mode_outlined),
              title: const Text('Tema gelap'),
              subtitle: const Text('Disimpan di SharedPreferences'),
              value: isDark,
              onChanged: (_) => ref.read(darkModeProvider.notifier).toggle(),
            ),
          ),
          ListTile(
            leading: const Icon(Icons.history),
            title: const Text('Terakhir dibuka'),
            subtitle: Text(
              lastOpenedAsync.when(
                data: (value) => value ?? 'Baru pertama kali dibuka',
                loading: () => 'Memuat...',
                error: (e, _) => 'Error: $e',
              ),
            ),
          ),
        ],
      ),
    );
  }
}

```

Langkah 4 Ganti main.dart

lib/main.dart [GANTI SELURUH ISI]

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'pages/settings_page.dart';
import 'providers/prefs_providers.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ProviderScope(child: OfflineNotesApp()));
}

class OfflineNotesApp extends ConsumerWidget {
  const OfflineNotesApp({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final isDark = ref.watch(darkModeProvider).value ?? false;

    return MaterialApp(
      title: 'Offline Notes',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        brightness: Brightness.light,
      ),
      darkTheme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        brightness: Brightness.dark,
      ),
      themeMode: isDark ? ThemeMode.dark : ThemeMode.light,
      home: const SettingsPage(), // diganti NotesPage pada Praktikum 3
    );
  }
}
```

Langkah 5 Jalankan dan uji

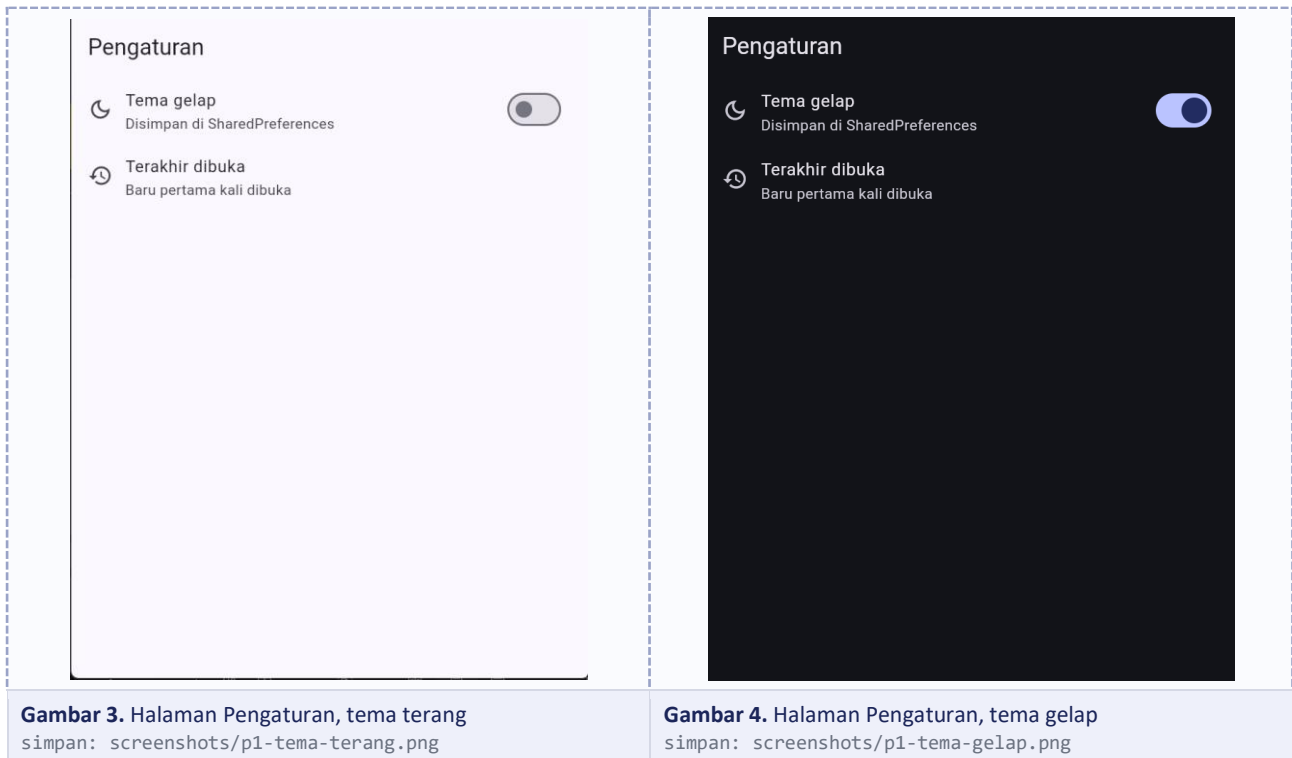
Terminal [TERMINAL]

```
flutter run
```

Kalau error

`MissingPluginException` → **stop aplikasi**, lalu jalankan ulang `flutter run`. Hot reload / hot restart tidak cukup setelah menambah plugin.

No	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
1	Ganti tema	Aktifkan switch Tema gelap	Tema langsung berubah gelap	Tema langsung berubah gelap	☐ Lulus
2	Persistensi tema	Tutup aplikasi total (swipe dari recent apps), buka lagi	Tema gelap masih aktif	Tema gelap masih aktif	☐ Lulus
3	Buka pertama	Lihat baris Terakhir dibuka saat pertama kali jalan	Tertulis "Baru pertama kali dibuka"	Tertulis "Baru pertama kali dibuka"	☐ Lulus
4	Buka berikutnya	Tutup total, buka lagi	Baris Terakhir dibuka berisi waktu sesi sebelumnya	Baris Terakhir dibuka berisi waktu sesi sebelumnya	☐ Lulus



Cek manual: tidak boleh ada pemanggilan SharedPreferences di folder `lib/pages/`.

```
Git Bash / macOS / Linux [TERMINAL]
grep -rn "shared_preferences" lib/pages
```

```
PowerShell (Windows) [TERMINAL]
Select-String -Path lib/pages/*.dart -Pattern "shared_preferences"
```

Hasil yang benar: **tidak ada output**.

Jawaban pertanyaan Praktikum 1

1.1 Mengapa `SharedPreferences.getInstance()` tidak boleh dipanggil di `build()`?

`build()` bisa dipanggil berkali-kali setiap widget rebuild dan harus sinkron. Memanggil operasi async ke platform di sana membuat pembacaan berulang, UI berkedip, dan sulit diuji. Kalau dibungkus repository + provider, hasil dibaca sekali, ter-cache, dan provider-nya bisa di-override saat test.

1.2 Alur data dari switch ditekan sampai tersimpan di disk dan tema berubah?

Switch ditekan → `onChanged` memanggil `toggle()` → `state` langsung diganti `AsyncData(next)` → `MaterialApp` yang me-watch `darkModeProvider` rebuild sehingga tema berubah → repository memanggil `setBool` lewat `SharedPreferences` → platform (Android) menulis ke berkas XML di disk. Saat aplikasi dibuka lagi, `build()` notifier membaca nilai itu dari disk.

1.3 Kelebihan dan risiko optimistic update pada `toggle()`?

Kelebihan: UI langsung responsif dan tidak berkedip karena tidak menunggu disk. Risiko: UI sempat menampilkan nilai yang ternyata gagal disimpan. Karena itu ada blok `catch` yang mengembalikan state ke nilai sebelumnya (rollback).

Commit praktikum ini:

Terminal [TERMINAL]

```
git add .  
git commit -m "feat(prefs): simpan tema dan waktu terakhir dibuka"
```

Praktikum 2 — Model & Database SQLite

Target: model *Note*, database *SQLite*, dan *NoteRepository* sebagai satu-satunya pintu data catatan.

Langkah 1 Model Note

lib/data/local/note.dart [BUAT FILE BARU]

```
class Note {
  const Note({
    this.id,
    required this.title,
    this.body = '',
    required this.updatedAt,
    this.dirty = false,
  });

  final int? id;
  final String title;
  final String body;
  final DateTime updatedAt;
  final bool dirty;

  Note copyWith({
    int? id,
    String? title,
    String? body,
    DateTime? updatedAt,
    bool? dirty,
  }) {
    return Note(
      id: id ?? this.id,
      title: title ?? this.title,
      body: body ?? this.body,
      updatedAt: updatedAt ?? this.updatedAt,
      dirty: dirty ?? this.dirty,
    );
  }

  Map<String, Object?> toMap() => {
    'id': id,
    'title': title,
    'body': body,
    'updated_at': updatedAt.toIso8601String(),
    'dirty': dirty ? 1 : 0,
  };

  factory Note.fromMap(Map<String, Object?> map) {
    return Note(
      id: (map['id'] as num?).toInt(),
      title: map['title'] as String? ?? '',
      body: map['body'] as String? ?? '',
      updatedAt: DateTime.tryParse(map['updated_at'] as String? ?? '') ??
        DateTime.fromMillisecondsSinceEpoch(0),
      dirty: ((map['dirty'] as num?).toInt() ?? 0) == 1,
    );
  }
}
```

Langkah 2 Pembuka database

lib/data/local/db.dart [BUAT FILE BARU]

```
import 'package:path/path.dart' as p;
import 'package:sqflite/sqflite.dart';

Future<Database> openNotesDb() async {
  final dir = await getDatabasesPath();
  return openDatabase(
```

```

p.join(dir, 'offline_notes.db'),
version: 1,
onCreate: (db, version) async {
  await db.execute('''
    CREATE TABLE notes(
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      title TEXT NOT NULL,
      body TEXT NOT NULL DEFAULT '',
      updated_at TEXT NOT NULL,
      dirty INTEGER NOT NULL DEFAULT 0
    )
  ''');
  await db.execute('''
    CREATE TABLE cached_posts(
      id INTEGER PRIMARY KEY,
      payload TEXT NOT NULL,
      cached_at TEXT NOT NULL
    )
  ''');
},
);
}

```

Langkah 3 NoteRepository

```

lib/data/repositories/note_repository.dart [BUAT FILE BARU]

import 'package:sqflite/sqflite.dart';

import '../local/db.dart';
import '../local/note.dart';

class NoteRepository {
  NoteRepository({Future<Database> Function()? openDb})
    : _openDb = openDb ?? openNotesDb;

  final Future<Database> Function() _openDb;

  Future<List<Note>> fetchNotes() async {
    final db = await _openDb();
    final rows = await db.query('notes', orderBy: 'updated_at DESC');
    return rows.map(Note.fromMap).toList();
  }

  Future<Note?> getNoteById(int id) async {
    final db = await _openDb();
    final rows = await db.query(
      'notes',
      where: 'id = ?',
      whereArgs: [id],
      limit: 1,
    );
    return rows.isEmpty ? null : Note.fromMap(rows.first);
  }

  Future<Note> addNote({required String title, String body = ''}) async {
    final db = await _openDb();
    final note = Note(
      title: title,
      body: body,
      updatedAt: DateTime.now(),
      dirty: true,
    );
    final id = await db.insert('notes', note.toMap());
    return note.copyWith(id: id);
  }

  Future<void> updateNote(Note note) async {
    if (note.id == null) {
      throw ArgumentError('Catatan belum memiliki id');
    }
  }
}

```

```

    }
    final db = await _openDb();
    final updated = note.copyWith(updatedAt: DateTime.now(), dirty: true);
    await db.update(
      'notes',
      updated.toMap(),
      where: 'id = ?',
      whereArgs: [note.id],
    );
  }

  Future<void> deleteNote(int id) async {
    final db = await _openDb();
    await db.delete('notes', where: 'id = ?', whereArgs: [id]);
  }

  Future<int> countDirty() async {
    final db = await _openDb();
    final rows = await db.rawQuery(
      'SELECT COUNT(*) AS c FROM notes WHERE dirty = 1',
    );
    return (rows.first['c'] as num?)?.toInt() ?? 0;
  }

  Future<void> markAllSynced() async {
    final db = await _openDb();
    await db.update('notes', {'dirty': 0}, where: 'dirty = 1');
  }
}

```

Langkah 4 Verifikasi dengan analyzer

Terminal [TERMINAL]

```
flutter analyze
```

Harus terjadi

Tidak ada error pada folder `lib/data/`. Praktikum ini belum mengubah tampilan aplikasi, jadi bukti hasilnya adalah output analyzer.

```

PS C:\laragon\www\244107020136-mobile-cflutter
analyze>
Analyzing 07-week-7...

No issues found! (ran in 4.4s)
PS C:\laragon\www\244107020136-mobile-course\07
-week-7>

```

Gambar 5. Output flutter analyze

simpan: screenshots/p2-analyze.png

Kalau error

`table notes already exists / no such column` → database lama masih ada di perangkat. Saat development, `uninstall aplikasi` dari emulator lalu `flutter run` lagi.

Jawaban pertanyaan Praktikum 2

2.1 Mengapa kolom `dirty` bertipe `INTEGER`, bukan `BOOLEAN`?

SQLite tidak punya tipe `BOOLEAN` asli. Nilai benar/salah disimpan sebagai `1/0`, lalu dikonversi di `toMap()` (`dirty ? 1 : 0`) dan `fromMap()` (`== 1`).

2.2 Apa fungsi parameter `openDb` pada constructor `NoteRepository`?

Itu dependency injection. Secara bawaan memakai `openNotesDb`, tetapi test bisa menyuntikkan database palsu atau fungsi yang melempar error sehingga repository bisa diuji tanpa SQLite sungguhan.

2.3 Mengapa query memakai `where: 'id = ?'` dan `whereArgs`, bukan interpolasi string?

Nilai dikirim terpisah sebagai parameter, sehingga aman dari SQL injection dan dari masalah escaping karakter khusus seperti tanda kutip.

2.4 Apa yang terjadi jika menambah kolom di `onCreate` tanpa menaikkan `version`?

`onCreate` hanya jalan saat berkas database belum ada. Di perangkat yang sudah punya database, kolom baru tidak pernah dibuat, sehingga muncul error `no such column`. Solusinya naikan `version` dan tambah `onUpgrade` (atau `uninstall aplikasi` saat development).

Commit praktikum ini:

Terminal [TERMINAL]

```
git add .
git commit -m "feat(db): model Note, skema SQLite, dan NoteRepository"
```

Praktikum 3 — Halaman Catatan Offline

Target: CRUD catatan dengan 4 state UI (loading, error, empty, success) yang tetap jalan di mode pesawat.

Langkah 1 Provider catatan

```
lib/providers/note_providers.dart [BUAT FILE BARU]

import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/local/note.dart';
import '../data/repositories/note_repository.dart';

final noteRepositoryProvider =
  Provider<NoteRepository>((ref) => NoteRepository());

final notesProvider = FutureProvider<List<Note>>{
  (ref) => ref.watch(noteRepositoryProvider).fetchNotes(),
};

final dirtyCountProvider = FutureProvider<int>{
  (ref) => ref.watch(noteRepositoryProvider).countDirty(),
};

final noteActionsProvider = Provider<NoteActions>((ref) => NoteActions(ref));

/// Kumpulan aksi yang mengubah data. Setiap mutasi diakhiri invalidate
/// agar provider baca (notes & dirty count) memuat ulang dari database.
class NoteActions {
  NoteActions(this._ref);

  final Ref _ref;

  NoteRepository get _repo => _ref.read(noteRepositoryProvider);

  Future<void> add(String title, String body) async {
    await _repo.addNote(title: title, body: body);
    _refresh();
  }

  Future<void> update(Note note) async {
    await _repo.updateNote(note);
    _refresh();
  }

  Future<void> delete(int id) async {
    await _repo.deleteNote(id);
    _refresh();
  }

  void _refresh() {
    _ref.invalidate(notesProvider);
    _ref.invalidate(dirtyCountProvider);
  }
}
```

Langkah 2 Dialog form catatan

```
lib/widgets/note_form_dialog.dart [BUAT FILE BARU]

import 'package:flutter/material.dart';

import '../data/local/note.dart';

typedef NoteFormResult = ({String title, String body});
```

```

class NoteFormDialog extends StatefulWidget {
  const NoteFormDialog({super.key, this.initial});

  final Note? initial;

  @override
  State<NoteFormDialog> createState() => _NoteFormDialogState();
}

class _NoteFormDialogState extends State<NoteFormDialog> {
  late final TextEditingController _title =
    TextEditingController(text: widget.initial?.title ?? '');
  late final TextEditingController _body =
    TextEditingController(text: widget.initial?.body ?? '');
  String? _error;

  @override
  void dispose() {
    _title.dispose();
    _body.dispose();
    super.dispose();
  }

  void _submit() {
    final title = _title.text.trim();
    if (title.isEmpty) {
      setState(() => _error = 'Judul wajib diisi');
      return;
    }
    Navigator.of(context).pop<NoteFormResult>(
      (title: title, body: _body.text.trim()),
    );
  }

  @override
  Widget build(BuildContext context) {
    return AlertDialog(
      title: Text(widget.initial == null ? 'Catatan baru' : 'Ubah catatan'),
      content: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          TextField(
            controller: _title,
            autofocus: true,
            decoration: InputDecoration(labelText: 'Judul', errorText: _error),
          ),
          const SizedBox(height: 8),
          TextField(
            controller: _body,
            maxLines: 4,
            decoration: const InputDecoration(labelText: 'Isi catatan'),
          ),
        ],
      ),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(context).pop(),
          child: const Text('Batal'),
        ),
        FilledButton(onPressed: _submit, child: const Text('Simpan')),
      ],
    );
  }
}

```

Langkah 3 Halaman catatan

lib/pages/notes_page.dart [BUAT FILE BARU]

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/local/note.dart';
import '../providers/note_providers.dart';
import '../widgets/note_form_dialog.dart';
import 'settings_page.dart';

class NotesPage extends ConsumerWidget {
  const NotesPage({super.key});

  Future<void> _openForm(
    BuildContext context,
    WidgetRef ref, [
    Note? note,
  ]) async {
    final result = await showDialog<NoteFormResult>(
      context: context,
      builder: (_) => NoteFormDialog(initial: note),
    );
    if (result == null) return; // dibatalkan
    final actions = ref.read(noteActionsProvider);
    if (note == null) {
      await actions.add(result.title, result.body);
    } else {
      await actions.update(
        note.copyWith(title: result.title, body: result.body),
      );
    }
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final notesAsync = ref.watch(notesProvider);
    final dirty = ref.watch(dirtyCountProvider).value ?? 0;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Offline Notes'),
        actions: [
          Tooltip(
            message: 'Catatan belum tersinkron',
            child: Badge(
              isLabelVisible: dirty > 0,
              label: Text('$dirty'),
              child: const Icon(Icons.cloud_upload_outlined),
            ),
          ),
          IconButton(
            tooltip: 'Pengaturan',
            icon: const Icon(Icons.settings_outlined),
            onPressed: () => Navigator.of(context).push(
              MaterialPageRoute(builder: (_) => const SettingsPage()),
            ),
          ),
        ],
      ),
      body: notesAsync.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => _ErrorView(
          message: 'Gagal membaca database: $e',
          onRetry: () => ref.invalidate(notesProvider),
        ),
        data: (notes) {
          if (notes.isEmpty) return const _EmptyView();
          return ListView.separated(
            itemCount: notes.length,
            separatorBuilder: (context, index) => const Divider(height: 1),
            itemBuilder: (context, index) {
              final note = notes[index];
```

```

        return ListTile(
          leading: Icon(
            note.dirty ? Icons.cloud_off : Icons.cloud_done,
            color: note.dirty ? Colors.orange : Colors.green,
          ),
          title: Text(note.title),
          subtitle: Text(
            note.body.isEmpty ? '(tanpa isi)' : note.body,
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
          ),
          onTap: () => _openForm(context, ref, note),
          trailing: IconButton(
            tooltip: 'Hapus',
            icon: const Icon(Icons.delete_outline),
            onPressed: () =>
              ref.read(noteActionsProvider).delete(note.id!),
          ),
        );
      },
    );
  },
),
floatingActionButton: FloatingActionButton.extended(
  onPressed: () => _openForm(context, ref),
  icon: const Icon(Icons.add),
  label: const Text('Catatan'),
),
);
}
}

class _EmptyView extends StatelessWidget {
  const _EmptyView();

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Icon(Icons.note_alt_outlined, size: 64),
          SizedBox(height: 12),
          Text('Belum ada catatan. Tekan + untuk menambah.'),
        ],
      ),
    );
  }
}

class _ErrorView extends StatelessWidget {
  const _ErrorView({required this.message, required this.onRetry});

  final String message;
  final VoidCallback onRetry;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            const Icon(Icons.error_outline, size: 48),
            const SizedBox(height: 12),
            Text(message, textAlign: TextAlign.center),
            const SizedBox(height: 12),
            FilledButton(onPressed: onRetry, child: const Text('Coba lagi')),
          ],
        ),
      ),
    );
  }
}

```

```
}
```

Langkah 4 Jadikan NotesPage halaman utama

Buka `lib/main.dart`, lalu lakukan **dua perubahan**:

`lib/main.dart` (perubahan 1: import) [\[UBAH\]](#)

```
// GANTI baris ini:  
import 'pages/settings_page.dart';  
// MENJADI:  
import 'pages/notes_page.dart';
```

`lib/main.dart` (perubahan 2: home) [\[UBAH\]](#)

```
// GANTI baris ini:  
home: const SettingsPage(), // diganti NotesPage pada Praktikum 3  
// MENJADI:  
home: const NotesPage(),
```

Langkah 5 Jalankan dan uji CRUD

Terminal [\[TERMINAL\]](#)

```
flutter run
```

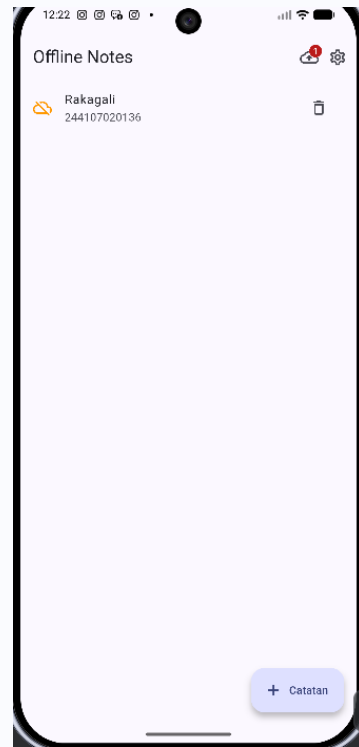
No	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
1	Empty state	Jalankan aplikasi pertama kali	Ikon + teks "Belum ada catatan"	Ikon + teks "Belum ada catatan"	☐ Lulus
2	Create	Tekan + Catatan , isi judul, Simpan	Catatan muncul paling atas, ikon awan oranye, badge = 1	Catatan muncul paling atas, ikon awan oranye, badge = 1	☐ Lulus
3	Validasi	Simpan dengan judul kosong	Muncul "Judul wajib diisi", dialog tidak tertutup	Muncul "Judul wajib diisi", dialog tidak tertutup	☐ Lulus
4	Update	Ketuk catatan, ubah isi, Simpan	Isi berubah, catatan naik ke posisi teratas	Isi berubah, catatan naik ke posisi teratas	☐ Lulus
5	Delete	Tekan ikon tempat sampah	Catatan hilang, badge berkurang bila catatan itu dirty	Catatan hilang, badge berkurang bila catatan itu dirty	☐ Lulus
6	Persistensi	Tutup aplikasi total, buka lagi	Semua catatan masih ada	Semua catatan masih ada	☐ Lulus
7	Mode pesawat	Aktifkan mode pesawat, ulangi skenario 2, 4, 5	Semua tetap jalan tanpa error	Semua tetap jalan tanpa error	☐ Lulus

Screenshot yang diambil (kotak ke-4 wajib: ikon pesawat harus terlihat di status bar):



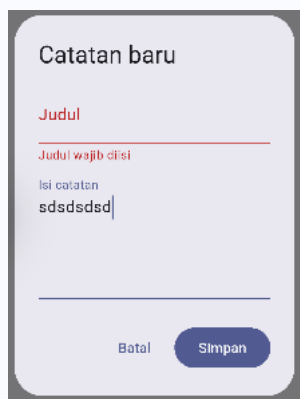
Gambar 6. Empty state

simpan: screenshots/p3-empty.png



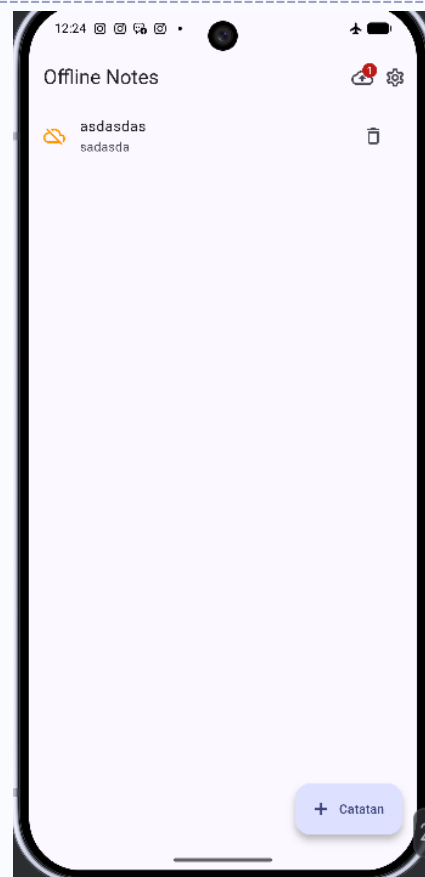
Gambar 7. Catatan baru + badge 1

simpan: screenshots/p3-tambah.png



Gambar 8. Validasi judul kosong

simpan: screenshots/p3-validasi.png



Gambar 9. Daftar catatan saat mode pesawat aktif

simpan: screenshots/p3-mode-pesawat.png

Cek manual: tidak boleh ada import `sqlite` atau `shared_preferences` di `lib/pages` dan `lib/widgets`.

Git Bash / macOS / Linux [TERMINAL]

```
grep -rE "sqlite|shared_preferences" lib/pages lib/widgets
```

PowerShell (Windows) [TERMINAL]

```
Select-String -Path lib/pages/*.dart, lib/widgets/*.dart -Pattern "sqlite|shared_preferences"
```

Hasil yang benar: **tidak ada output**.

Jawaban pertanyaan Praktikum 3

3.1 Mengapa setelah mutasi perlu `invalidate notesProvider` dan `dirtyCountProvider`? Apa yang terjadi jika hanya satu?

Keduanya membaca tabel yang sama dari sudut berbeda. Jika hanya `notesProvider` yang di-invalidate, daftar berubah tetapi badge dirty basi. Jika hanya `dirtyCountProvider`, badge benar tetapi daftar tidak ikut berubah.

3.2 Bagaimana memicu state error secara sengaja untuk menguji `_ErrorView`?

Sementara ubah nama tabel di `fetchNotes()` (misalnya `'notes'` menjadi `'notess'`) lalu jalankan: query gagal dan `_ErrorView` tampil. Cara lain: melempar `Exception` manual di `fetchNotes()`, atau meng-override `noteRepositoryProvider` dengan repository palsu yang melempar error (seperti di Praktikum 5). Kembalikan kodenya setelah uji.

3.3 Mengapa aplikasi tetap jalan di mode pesawat tanpa kode khusus offline?

Semua baca/tulis catatan menuju SQLite lokal di perangkat. Tidak ada satu pun operasi catatan yang membutuhkan jaringan, jadi ada atau tidaknya internet tidak berpengaruh.

Commit praktikum ini:

Terminal [TERMINAL]

```
git add .
git commit -m "feat(notes): halaman catatan offline dengan Riverpod"
```

Praktikum 4 — Cache-First & Sinkronisasi

Target: dirty flag + sync (server simulasi), saklar offline, dan cache posts. Butuh internet SEKALI saat pertama membuka halaman Posts.

Langkah 1 Saklar offline

lib/providers/offline_providers.dart [BUAT FILE BARU]

```
import 'package:flutter_riverpod/flutter_riverpod.dart';

/// Simulasi offline yang deterministik untuk demo dan testing,
/// sehingga tidak bergantung pada kondisi Wi-Fi kelas.
final forceOfflineProvider =
  NotifierProvider<ForceOfflineNotifier, bool>(ForceOfflineNotifier.new);

class ForceOfflineNotifier extends Notifier<bool> {
  @override
  bool build() => false;

  void toggle() => state = !state;
}
```

Langkah 2 Logika sinkronisasi

lib/data/sync.dart [BUAT FILE BARU]

```
import 'local/note.dart';
import 'repositories/note_repository.dart';

class OfflineException implements Exception {
  const OfflineException(this.message);

  final String message;

  @override
  String toString() => 'OfflineException: $message';
}

/// Mengirim catatan dirty ke "server" (simulasi) lalu menandainya bersih.
/// Mengembalikan jumlah catatan yang tersinkron.
Future<int> syncNotes(
  NoteRepository repo, {
  bool offline = false,
  Duration latency = const Duration(seconds: 1),
}) async {
  if (offline) {
    throw const OfflineException('Perangkat offline, sinkronisasi ditunda.');
```

```
  }
  final dirtyCount = await repo.countDirty();
  if (dirtyCount == 0) return 0;

  // Simulasi upload. Pada project nyata: kirim tiap catatan dirty
  // ke REST API, lalu tandai bersih HANYA bila server menjawab 2xx.
  await Future.delayed(latency);
  await repo.markAllSynced();
  return dirtyCount;
}

/// Aturan konflik: last-write-wins berdasarkan updatedAt.
Note resolveConflict(Note local, Note remote) {
  return remote.updatedAt.isAfter(local.updatedAt) ? remote : local;
}
```

Langkah 3 Tambah aksi sync() di note_providers.dart

Isi file ini diganti dengan versi lengkap di bawah (sudah memuat import baru dan method `sync()`):

```
lib/providers/note_providers.dart [GANTI SELURUH ISI]

import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/local/note.dart';
import '../data/repositories/note_repository.dart';
import '../data/sync.dart';
import 'offline_providers.dart';

final noteRepositoryProvider =
  Provider<NoteRepository>((ref) => NoteRepository());

final notesProvider = FutureProvider<List<Note>>((
  ref) => ref.watch(noteRepositoryProvider).fetchNotes(),
);

final dirtyCountProvider = FutureProvider<int>((
  ref) => ref.watch(noteRepositoryProvider).countDirty(),
);

final noteActionsProvider = Provider<NoteActions>((ref) => NoteActions(ref));

/// Kumpulan aksi yang mengubah data. Setiap mutasi diakhiri invalidate
/// agar provider baca (notes & dirty count) memuat ulang dari database.
class NoteActions {
  NoteActions(this._ref);

  final Ref _ref;

  NoteRepository get _repo => _ref.read(noteRepositoryProvider);

  Future<void> add(String title, String body) async {
    await _repo.addNote(title: title, body: body);
    _refresh();
  }

  Future<void> update(Note note) async {
    await _repo.updateNote(note);
    _refresh();
  }

  Future<void> delete(int id) async {
    await _repo.deleteNote(id);
    _refresh();
  }

  Future<int> sync() async {
    final offline = _ref.read(forceOfflineProvider);
    final count = await syncNotes(_repo, offline: offline);
    _refresh();
    return count;
  }

  void _refresh() {
    _ref.invalidate(notesProvider);
    _ref.invalidate(dirtyCountProvider);
  }
}
```

Langkah 4 Model Post

lib/data/remote/post.dart [BUAT FILE BARU]

```
class Post {
  const Post({required this.id, required this.title, required this.body});

  final int id;
  final String title;
  final String body;

  factory Post.fromJson(Map<String, dynamic> json) => Post(
    id: (json['id'] as num).toInt(),
    title: json['title'] as String? ?? '',
    body: json['body'] as String? ?? '',
  );

  Map<String, dynamic> toJson() => {'id': id, 'title': title, 'body': body};
}
```

Langkah 5 Repository posts dengan cache

lib/data/repositories/post_repository.dart [BUAT FILE BARU]

```
import 'dart:convert';

import 'package:dio/dio.dart';
import 'package:sqflite/sqflite.dart';

import '../local/db.dart';
import '../remote/post.dart';

class PostRepository {
  PostRepository({Dio? dio, Future<Database> Function()? openDb})
    : _dio = dio ??
      Dio(BaseOptions(
        baseUrl: 'https://jsonplaceholder.typicode.com',
        connectTimeout: const Duration(seconds: 5),
        receiveTimeout: const Duration(seconds: 5),
      )),
      _openDb = openDb ?? openNotesDb;

  final Dio _dio;
  final Future<Database> Function() _openDb;

  /// Baca cache dari tabel cached_posts (tanpa jaringan).
  Future<List<Post>> readCachedPosts() async {
    final db = await _openDb();
    final rows = await db.query('cached_posts', orderBy: 'id ASC');
    return rows
      .map((r) => Post.fromJson(
        jsonDecode(r['payload'] as String) as Map<String, dynamic>,
      ))
      .toList();
  }

  /// Ambil dari jaringan, lalu ganti isi cache dalam satu transaksi.
  Future<List<Post>> fetchAndCache() async {
    final res = await _dio.get<List<dynamic>>('/posts');
    final posts = (res.data ?? const [])
      .map((e) => Post.fromJson(e as Map<String, dynamic>))
      .toList();

    final db = await _openDb();
    final now = DateTime.now().toIso8601String();
    await db.transaction((txn) async {
      await txn.delete('cached_posts');
      final batch = txn.batch();
    });
  }
}
```

```

    for (final post in posts) {
      batch.insert('cached_posts', {
        'id': post.id,
        'payload': jsonEncode(post.toJson()),
        'cached_at': now,
      });
    }
    await batch.commit(noResult: true);
  });
  return posts;
}
}

```

Langkah 6 Provider cache-first

lib/providers/post_providers.dart [BUAT FILE BARU]

```

import 'dart:async';

import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/remote/post.dart';
import '../data/repositories/post_repository.dart';
import '../data/sync.dart';
import 'offline_providers.dart';

final postRepositoryProvider =
  Provider<PostRepository>((ref) => PostRepository());

final postsProvider =
  AsyncNotifierProvider<PostsNotifier, List<Post>>(PostsNotifier.new);

class PostsNotifier extends AsyncNotifier<List<Post>> {
  @override
  Future<List<Post>> build() async {
    final repo = ref.watch(postRepositoryProvider);
    final offline = ref.watch(forceOfflineProvider);

    // 1. Selalu baca cache lokal terlebih dahulu.
    final cached = await repo.readCachedPosts();

    if (offline) {
      if (cached.isEmpty) {
        throw const OfflineException(
          'Offline dan belum ada cache. Buka halaman ini sekali saat online.',
        );
      }
      return cached;
    }

    // 2. Belum ada cache: terpaksa menunggu jaringan.
    if (cached.isEmpty) return repo.fetchAndCache();

    // 3. Ada cache: tampilkan segera, refresh di background.
    unawaited(_refreshInBackground(repo));
    return cached;
  }

  Future<void> _refreshInBackground(PostRepository repo) async {
    try {
      final fresh = await repo.fetchAndCache();
      state = AsyncData(fresh);
    } catch (_) {
      // Jaringan gagal: tetap tampilkan cache, jangan ganti UI dengan error.
    }
  }

  /// Dipanggil oleh pull-to-refresh. Mengembalikan true bila berhasil.

```

```

Future<bool> refresh() async {
  if (ref.read(forceOfflineProvider)) return false;
  try {
    final fresh = await ref.read(postRepositoryProvider).fetchAndCache();
    state = AsyncData(fresh);
    return true;
  } catch (_) {
    return false; // data lama (cache) tetap ditampilkan
  }
}
}

```

Langkah 7 Halaman posts

lib/pages/posts_page.dart [BUAT FILE BARU]

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../providers/offline_providers.dart';
import '../providers/post_providers.dart';

class PostsPage extends ConsumerWidget {
  const PostsPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final postsAsync = ref.watch(postsProvider);
    final offline = ref.watch(forceOfflineProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Posts (cache-first)'),),
      body: Column(
        children: [
          if (offline)
            Container(
              width: double.infinity,
              color: Colors.orange.shade100,
              padding: const EdgeInsets.all(12),
              child: const Text(
                'Mode offline: menampilkan data dari cache lokal',
                style: TextStyle(color: Colors.black87),
              ),
            ),
          Expanded(
            child: postsAsync.when(
              loading: () => const Center(child: CircularProgressIndicator()),
              error: (e, _) => Center(
                child: Padding(
                  padding: const EdgeInsets.all(24),
                  child: Text('$e', textAlign: TextAlign.center),
                ),
              ),
              data: (posts) => RefreshIndicator(
                onRefresh: () async {
                  final ok = await ref.read(postsProvider.notifier).refresh();
                  if (!ok && context.mounted) {
                    ScaffoldMessenger.of(context).showSnackBar(
                      const SnackBar(
                        content: Text('Gagal refresh, menampilkan cache'),
                      ),
                    );
                  }
                },
              ),
            child: ListView.builder(
              itemCount: posts.length,
              itemBuilder: (context, index) {
                final post = posts[index];
                return ListTile(
                  leading: CircleAvatar(child: Text(post.id.toString())),
                  title: Text(post.title, maxLines: 1),
                );
              },
            ),
          ),
        ],
      ),
    );
  }
}

```

```

        subtitle: Text(post.body, maxLines: 2),
      ),
    ),
  ),
),
),
],
),
);
}
}

```

Langkah 8 Tombol Posts dan Sinkronkan di NotesPage

Isi file diganti versi lengkap. Bedanya dari Praktikum 3: dua import baru dan dua `IconButton` (Posts, Sinkronkan) sebelum tombol Pengaturan.

```

lib/pages/notes_page.dart    [GANTI SELURUH ISI]

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/local/note.dart';
import '../data/sync.dart';
import '../providers/note_providers.dart';
import '../widgets/note_form_dialog.dart';
import 'posts_page.dart';
import 'settings_page.dart';

class NotesPage extends ConsumerWidget {
  const NotesPage({super.key});

  Future<void> _openForm(
    BuildContext context,
    WidgetRef ref, [
    Note? note,
  ]) async {
    final result = await showDialog<NoteFormResult>(
      context: context,
      builder: (_) => NoteFormDialog(initial: note),
    );
    if (result == null) return; // dibatalkan
    final actions = ref.read(noteActionsProvider);
    if (note == null) {
      await actions.add(result.title, result.body);
    } else {
      await actions.update(
        note.copyWith(title: result.title, body: result.body),
      );
    }
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final notesAsync = ref.watch(notesProvider);
    final dirty = ref.watch(dirtyCountProvider).value ?? 0;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Offline Notes'),
        actions: [
          Tooltip(
            message: 'Catatan belum tersinkron',
            child: Badge(
              isLabelVisible: dirty > 0,
              label: Text('$dirty'),
              child: const Icon(Icons.cloud_upload_outlined),
            ),
          ),
        ],
      ),
    );
  }
}

```

```

IconButton(
  tooltip: 'Posts (cache-first)',
  icon: const Icon(Icons.article_outlined),
  onPressed: () => Navigator.of(context).push(
    MaterialPageRoute(builder: (_) => const PostsPage()),
  ),
),
IconButton(
  tooltip: 'Sinkronkan',
  icon: const Icon(Icons.sync),
  onPressed: () async {
    final messenger = ScaffoldMessenger.of(context);
    try {
      final count = await ref.read(noteActionsProvider).sync();
      messenger.showSnackBar(SnackBar(
        content: Text(count == 0
          ? 'Semua catatan sudah tersinkron'
          : '$count catatan berhasil disinkronkan'),
      ));
    } on OfflineException catch (e) {
      messenger.showSnackBar(SnackBar(content: Text(e.message)));
    }
  },
),
IconButton(
  tooltip: 'Pengaturan',
  icon: const Icon(Icons.settings_outlined),
  onPressed: () => Navigator.of(context).push(
    MaterialPageRoute(builder: (_) => const SettingsPage()),
  ),
),
],
),
body: notesAsync.when(
  loading: () => const Center(child: CircularProgressIndicator()),
  error: (e, _) => _ErrorView(
    message: 'Gagal membaca database: $e',
    onRetry: () => ref.invalidate(notesProvider),
  ),
),
data: (notes) {
  if (notes.isEmpty) return const _EmptyView();
  return ListView.separated(
    itemCount: notes.length,
    separatorBuilder: (context, index) => const Divider(height: 1),
    itemBuilder: (context, index) {
      final note = notes[index];
      return ListTile(
        leading: Icon(
          note.dirty ? Icons.cloud_off : Icons.cloud_done,
          color: note.dirty ? Colors.orange : Colors.green,
        ),
        title: Text(note.title),
        subtitle: Text(
          note.body.isEmpty ? '(tanpa isi)' : note.body,
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
        ),
        onTap: () => _openForm(context, ref, note),
        trailing: IconButton(
          tooltip: 'Hapus',
          icon: const Icon(Icons.delete_outline),
          onPressed: () =>
            ref.read(noteActionsProvider).delete(note.id!),
        ),
      );
    },
  );
},
),
floatingActionButton: FloatingActionButton.extended(
  onPressed: () => _openForm(context, ref),
  icon: const Icon(Icons.add),
  label: const Text('Catatan'),
),
);
}

```

```

}

class _EmptyView extends StatelessWidget {
  const _EmptyView();

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Icon(Icons.note_alt_outlined, size: 64),
          SizedBox(height: 12),
          Text('Belum ada catatan. Tekan + untuk menambah.'),
        ],
      ),
    );
  }
}

class _ErrorView extends StatelessWidget {
  const _ErrorView({required this.message, required this.onRetry});

  final String message;
  final VoidCallback onRetry;

  @override
  Widget build(BuildContext context) {
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            const Icon(Icons.error_outline, size: 48),
            const SizedBox(height: 12),
            Text(message, textAlign: TextAlign.center),
            const SizedBox(height: 12),
            FilledButton(onPressed: onRetry, child: const Text('Coba lagi')),
          ],
        ),
      ),
    );
  }
}

```

Langkah 9 Saklar offline di SettingsPage

Isi file diganti versi lengkap. Bedanya: import `offline_providers.dart`, variabel `offline`, dan `SwitchListTile` Paksa mode offline di akhir `ListView`.

```

lib/pages/settings_page.dart [GANTI SELURUH ISI]

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../providers/offline_providers.dart';
import '../providers/prefs_providers.dart';

class SettingsPage extends ConsumerWidget {
  const SettingsPage({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final darkAsync = ref.watch(darkModeProvider);
    final lastOpenedAsync = ref.watch(lastOpenedProvider);
    final offline = ref.watch(forceOfflineProvider);

    return Scaffold(

```

```

appBar: AppBar(title: const Text('Pengaturan')),
body: ListView(
  children: [
    darkAsync.when(
      loading: () => const ListTile(
        title: Text('Memuat preferensi...'),
        trailing: SizedBox(
          width: 24,
          height: 24,
          child: CircularProgressIndicator(strokeWidth: 2),
        ),
      ),
      error: (e, _) => ListTile(
        leading: const Icon(Icons.error_outline),
        title: const Text('Gagal memuat preferensi'),
        subtitle: Text('$e'),
      ),
      data: (isDark) => SwitchListTile(
        secondary: const Icon(Icons.dark_mode_outlined),
        title: const Text('Tema gelap'),
        subtitle: const Text('Disimpan di SharedPreferences'),
        value: isDark,
        onChanged: (_) => ref.read(darkModeProvider.notifier).toggle(),
      ),
    ),
    ListTile(
      leading: const Icon(Icons.history),
      title: const Text('Terakhir dibuka'),
      subtitle: Text(
        lastOpenedAsync.when(
          data: (value) => value ?? 'Baru pertama kali dibuka',
          loading: () => 'Memuat...',
          error: (e, _) => 'Error: $e',
        ),
      ),
    ),
    SwitchListTile(
      secondary: const Icon(Icons.wifi_off),
      title: const Text('Paksa mode offline'),
      subtitle: const Text('Simulasi offline untuk demo dan testing'),
      value: offline,
      onChanged: (_) =>
        ref.read(forceOfflineProvider.notifier).toggle(),
    ),
  ],
),
);
}

```

Langkah 10 Uji offline-first

Stop aplikasi lalu **flutter run** ulang (ada file baru dan perubahan besar).

Terminal [TERMINAL]

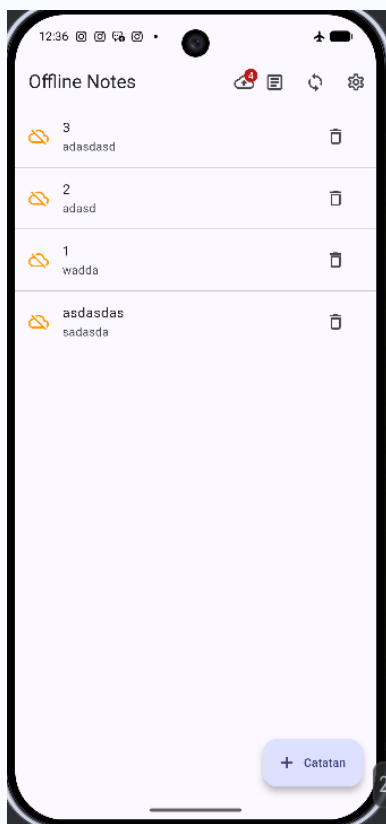
```
flutter run
```

No	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
1	Isi cache	Saat online , buka halaman Posts (ikon artikel)	100 posts tampil, tersimpan di cached_posts	100 posts tampil, tersimpan di cached_posts	☐ Lulus
2	Cache saat offline	Aktifkan mode pesawat, tutup lalu buka aplikasi, buka Posts	Posts tetap tampil dari cache	Posts tetap tampil dari cache	☐ Lulus
3	Antrean dirty	Masih offline, tambah 3 catatan	Badge menunjukkan angka 3	Badge menunjukkan angka 3	☐ Lulus

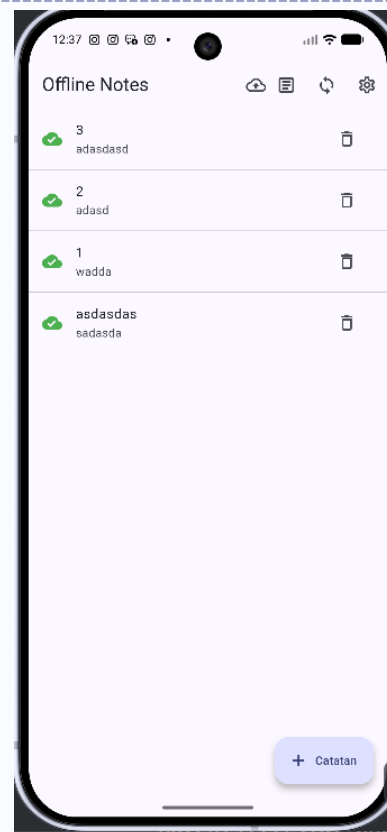
No	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
4	Sync ditolak	Settings → aktifkan Paksa mode offline , tekan ikon Sinkronkan	Snackbar "Perangkat offline...", badge tetap 3	Snackbar "Perangkat offline...", badge tetap 3	☐ Lulus
5	Sync berhasil	Matikan saklar offline, tekan Sinkronkan	Setelah ±1 detik snackbar "3 catatan berhasil disinkronkan", badge hilang, ikon awan hijau	Setelah ±1 detik snackbar "3 catatan berhasil disinkronkan", badge hilang, ikon awan hijau	☐ Lulus
6	Refresh background	Online, buka Posts	Daftar tampil seketika dari cache lalu diperbarui	Daftar tampil seketika dari cache lalu diperbarui	☐ Lulus

Urutan penting

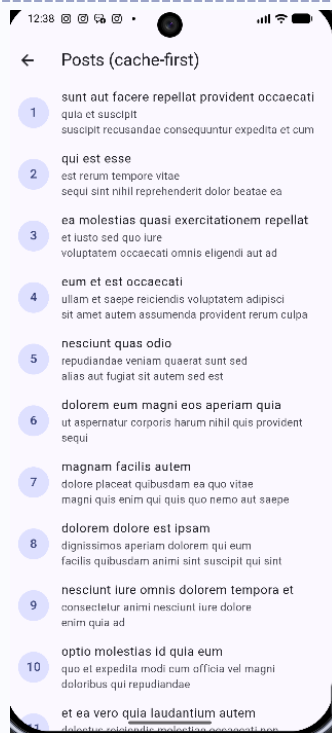
Skenario 1 **harus** dilakukan saat online lebih dulu, kalau tidak Posts error "Offline dan belum ada cache". Untuk screenshot skenario 3 sebelum sync, jangan tekan Sinkronkan dulu.



Gambar 10. Badge dirty sebelum sync (angka 3)
simpan: screenshots/p4-dirty-sebelum.png



Gambar 11. Badge hilang, ikon awan hijau setelah sync
simpan: screenshots/p4-dirty-sesudah.png



Gambar 12. Halaman Posts saat offline (banner oranye)

simpan: screenshots/p4-posts-offline.png

Langkah 11 Dokumentasi uji: docs/uji-offline.md

Buat file ini, lalu isi dengan hasil pengamatanmu sendiri:

docs/uji-offline.md [BUAT FILE BARU]

Uji Offline-First

Perangkat: (emulator / HP, versi Android)

Tanggal uji:

No	Skenario	Hasil pengamatan
1	Isi cache posts saat online	
2	Posts saat mode pesawat	
3	Tambah 3 catatan offline (badge)	
4	Sync saat Paksa mode offline	
5	Sync saat online	
6	Refresh background posts	

Aturan konflik yang dipakai: last-write-wins (resolveConflict, updated_at terbaru).

Aturan konflik yang dipakai

Last-write-wins berdasarkan `updated_at`: versi dengan waktu paling baru yang dipertahankan (fungsi `resolveConflict` di `sync.dart`). Catat juga di README.

Jawaban pertanyaan Praktikum 4

4.1 Beda cache-first dan network-first? Contoh data yang lebih cocok network-first?

Cache-first: tampilkan data lokal dulu, lalu perbarui dari jaringan di background. Cocok untuk artikel atau daftar posts.

Network-first: coba jaringan dulu, cache hanya cadangan jika gagal. Lebih cocok untuk data yang tidak boleh basi, misalnya harga saham, saldo rekening, atau stok real-time.

4.2 Skenario kehilangan data akibat `markAllSynced()` dan perbaikannya?

`markAllSynced()` menandai **semua** catatan dirty menjadi bersih. Jika pengguna mengedit catatan saat upload sedang berjalan (± 1 detik), versi barunya ikut ditandai bersih padahal belum terkirim, sehingga tidak pernah disinkronkan.

Perbaikan: tandai bersih hanya untuk catatan yang benar-benar dikirim, misalnya `WHERE id = ? AND updated_at = ?` berdasarkan daftar id + `updated_at` yang dikirim, atau pakai tabel outbox.

4.3 Mengapa perlu saklar `forceOffline` padahal sudah ada mode pesawat?

Agar demo dan test deterministik: tidak bergantung pada Wi-Fi kelas, bisa diotomatisasi, dan tidak perlu mengubah pengaturan perangkat. Cukup satu switch di aplikasi.

4.4 Mengapa `fetchAndCache()` menulis cache di dalam transaksi?

Hapus cache lama + insert data baru menjadi satu kesatuan atomik. Jika gagal di tengah jalan, cache lama tetap utuh dan tidak tersisa cache setengah jadi.

Commit praktikum ini:

Terminal [TERMINAL]

```
git add .
git commit -m "feat(sync): cache-first posts, dirty flag, dan sinkronisasi"
```

Praktikum 5 — Testing dengan Repository Palsu

Target: menguji model, provider, dan sync tanpa database sungguhan. Semua test lulus.

Langkah 1 Hapus test bawaan

`widget_test.dart` menguji aplikasi counter yang sudah tidak ada, jadi pasti gagal.

Git Bash / macOS / Linux [TERMINAL]

```
rm test/widget_test.dart
```

PowerShell (Windows) [TERMINAL]

```
del test/widget_test.dart
```

Langkah 2 Buat file test

test/note_test.dart [BUAT FILE BARU]

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:week5_offline_notes/data/local/note.dart';
import 'package:week5_offline_notes/data/repositories/note_repository.dart';
import 'package:week5_offline_notes/data/sync.dart';
import 'package:week5_offline_notes/providers/note_providers.dart';

class FakeNoteRepository extends NoteRepository {
  FakeNoteRepository({List<Note> items = const [], this.throwError = false})
    : items = [...items],
      super(openDb: () => throw UnimplementedError());

  final List<Note> items;
  final bool throwError;

  @override
  Future<List<Note>> fetchNotes() async {
    if (throwError) throw Exception('db locked (simulasi)');
    return items;
  }

  @override
  Future<int> countDirty() async => items.where((n) => n.dirty).length;

  @override
  Future<void> markAllSynced() async {
    for (var i = 0; i < items.length; i++) {
      items[i] = items[i].copyWith(dirty: false);
    }
  }
}

Note _note(String title, {bool dirty = false, DateTime? at}) =>
  Note(title: title, updatedAt: at ?? DateTime(2026, 9, 18), dirty: dirty);

void main() {
  group('Model Note', () {
    test('fromMap aman terhadap field yang hilang', () {
      final note = Note.fromMap({'title': 'Belanja'});
      expect(note.title, 'Belanja');
      expect(note.body, '');
      expect(note.dirty, isFalse);
    });

    test('flag dirty bertahan pada serialisasi', () {
      final note = _note('a', dirty: true);
    });
  });
}
```

```

        final restored = Note.fromMap(note.toMap());
        expect(restored.dirty, isTrue);
        expect(restored.updatedAt, note.updatedAt);
    });
});

group('Provider dengan repository palsu', () {
    test('notesProvider sukses', () async {
        final container = ProviderContainer(
            overrides: [
                noteRepositoryProvider.overrideWithValue(
                    FakeNoteRepository(items: [_note('Tes')]),
                ),
            ],
        );
        addTearDown(container.dispose);

        final notes = await container.read(notesProvider.future);
        expect(notes.length, 1);
        expect(notes.first.title, 'Tes');
    });

    test('notesProvider error', () async {
        final container = ProviderContainer(
            overrides: [
                noteRepositoryProvider.overrideWithValue(
                    FakeNoteRepository(throwError: true),
                ),
            ],
        );
        addTearDown(container.dispose);

        await expectLater(
            container.read(notesProvider.future),
            throwsA(isA<Exception>()),
        );
    });
});

group('Offline-first', () {
    test('syncNotes membersihkan semua catatan dirty', () async {
        final repo = FakeNoteRepository(items: [
            _note('a', dirty: true),
            _note('b', dirty: true),
            _note('c'),
        ]);
        final synced = await syncNotes(repo, latency: Duration.zero);
        expect(synced, 2);
        expect(await repo.countDirty(), 0);
    });

    test('syncNotes ditolak saat offline dan dirty tetap utuh', () async {
        final repo = FakeNoteRepository(items: [_note('a', dirty: true)]);
        await expectLater(
            syncNotes(repo, offline: true),
            throwsA(isA<OfflineException>()),
        );
        expect(await repo.countDirty(), 1);
    });

    test('resolveConflict memilih updatedAt terbaru', () {
        final local = _note('lokal', at: DateTime(2026, 9, 18, 10));
        final remote = _note('server', at: DateTime(2026, 9, 18, 11));
        expect(resolveConflict(local, remote).title, 'server');
        expect(resolveConflict(remote, local).title, 'server');
    });
});
}

```

Langkah 3 Jalankan analyze dan test

Terminal [TERMINAL]

```
flutter analyze
flutter test
```

Harus terjadi

No issues found! untuk analyze, dan All tests passed! (7 test) untuk flutter test.

Kalau test provider error berjalan sangat lama (Riverpod 3)

Penyebabnya fitur automatic retry. Ubah pembuatan container pada test "notesProvider error" menjadi:

```
ProviderContainer(retry: (count, error) => null, overrides: [ ... ])
```

```
00:02 +0 -1: Some tests failed.
PS C:\laragon\www\244107020136-mobile-course\07-week-7\week5_offline_notes> flutter test
00:02 +7: All tests passed!
PS C:\laragon\www\244107020136-mobile-course\07-week-7\week5_offline_notes> flutter analyze
Analyzing week5_offline_notes...

No issues found! (ran in 4.8s)
PS C:\laragon\www\244107020136-mobile-course\07-week-7\week5_offline_notes>
```

Gambar 13. flutter analyze dan flutter test lulus

simpan: screenshots/p5-flutter-test.png

No	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
1	fromMap field hilang	flutter test	Lulus: tidak crash, nilai bawaan terisi	Lulus: tidak crash, nilai bawaan terisi	☐ Lulus
2	dirty bertahan	flutter test	Lulus: bool↔0/1 dan DateTime↔ISO tidak hilang	Lulus: bool↔0/1 dan DateTime↔ISO tidak hilang	☐ Lulus
3	notesProvider sukses	flutter test	Lulus: data repository palsu sampai ke provider	Lulus: data repository palsu sampai ke provider	☐ Lulus
4	notesProvider error	flutter test	Lulus: exception jadi state error	Lulus: exception jadi state error	☐ Lulus
5	syncNotes bersihkan dirty	flutter test	Lulus: 2 tersinkron, countDirty = 0	Lulus: 2 tersinkron, countDirty = 0	☐ Lulus
6	syncNotes ditolak offline	flutter test	Lulus: OfflineException, dirty tetap 1	Lulus: OfflineException, dirty tetap 1	☐ Lulus

No	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
7	resolveConflict	<code>flutter test</code>	Lulus: yang updatedAt terbaru menang	Lulus: yang updatedAt terbaru menang	☐ Lulus

Jawaban pertanyaan Praktikum 5

5.1 Mengapa provider diuji dengan `ProviderContainer` + `overrideWithValue`, bukan database asli?

Lebih cepat, deterministik, dan berjalan tanpa emulator/plugin. Fokus test ada pada logika provider, bukan pada SQLite. Test yang menyentuh SQLite asli lambat dan tidak stabil di lingkungan `flutter test`.

5.2 Apa manfaat parameter `latency` pada `syncNotes` untuk pengujian?

Test bisa memakai `Duration.zero` sehingga berjalan cepat, sementara aplikasi tetap memakai jeda simulasi 1 detik.

5.3 Satu test tambahan yang penting namun belum ada?

`updateNote` harus menandai `dirty = true` dan memperbarui `updated_at`. Alasannya: seluruh mekanisme antrean sync bergantung pada dua hal ini; jika salah, perubahan catatan tidak pernah terkirim ke server.

Commit praktikum ini:

Terminal [TERMINAL]

```
git add .
git commit -m "test: model, provider, dan sync dengan repository palsu"
```

AI Prompt Challenge

Bukti keputusan teknis. Simpan prompt, output awal AI, dan keputusan akhir di folder docs/.

Langkah 1 Jalankan prompt ini di AI assistant

Prompt [TERMINAL]

Aplikasi Flutter Offline Notes: CRUD catatan + preferensi tema. Bandingkan SharedPreferences, Hive, sqflite (SQLite), dan Drift untuk dua kebutuhan ini. Requirements:

- Kriteria: kompleksitas query, kebutuhan relasi, reaktivitas (stream), type-safety, ukuran boilerplate, dan kemudahan testing.
- Beri rekomendasi final: mana untuk preferensi, mana untuk catatan, beserta alasannya dalam 1 tabel.
- Tunjukkan skema tabel/kotak untuk 1000+ catatan. Jelaskan trade-off setiap pilihan.

Tempel ringkasan output awal AI di kotak ini (simpan lengkapnya ke docs/):

Ringkasan Output Komparasi Storage:

1. Matriks Perbandingan (6 Kriteria)

- **Kompleksitas Query:** `SharedPreferences` (sangat rendah, hanya key-value), `Hive` (sedang, filter in-memory), `sqflite` (tinggi, SQL penuh: WHERE, ORDER BY, LIMIT), `Drift` (sangat tinggi, SQL + type-safe fluent API).
- **Kebutuhan Relasi:** `SharedPreferences` (tidak ada), `Hive` (manual), `sqflite` (Foreign Key & JOIN relasional penuh), `Drift` (Foreign Key, JOIN, cascade delete otomatis).
- **Reaktivitas (Stream):** `SharedPreferences` (manual via Riverpod/StateNotifier), `Hive` (Stream watch box bawaan), `sqflite` (manual via invalidate / wrapper stream), `Drift` (native auto-updating Stream `watch()`).
- **Type-Safety:** `SharedPreferences` (lemah/primitif), `Hive` (sedang via TypeAdapter), `sqflite` (rendah, map & raw string), `Drift` (sangat tinggi, compile-time checked).
- **Ukuran Boilerplate:** `SharedPreferences` (sangat minim), `Hive` (sedang, butuh adapter/generator), `sqflite` (sedang-tinggi, manual SQL & mapper), `Drift` (tinggi setup awal + `build_runner`, minim saat penulisan query).

Gambar 14. Ringkasan output awal AI

simpan: docs/ai-output-awal.md

Langkah 2 Verifikasi hasil AI

Isi kolom **Temuan** setelah kamu mencoba sendiri. Kolom hint adalah acuan apa yang dicek.

Pemeriksaan	Hint	Temuan kamu
AI menaruh daftar catatan di SharedPreferences?	Jika ya, tolak: rapuh untuk koleksi	Tidak (Lolos). AI menolak keras penggunaan SharedPreferences untuk daftar catatan dan melabelinya sebagai <i>anti-pattern</i> . SharedPreferences hanya direkomendasikan untuk data primitif (tema dan waktu buka), sedangkan daftar catatan diarahkan ke <code>sqflite</code> / <code>Drift</code> karena menyimpan koleksi 1000 item dalam string JSON raksasa akan membebani CPU, memblokir thread, dan memicu ANR.

Pemeriksaan	Hint	Temuan kamu
Skema AI mendukung antrean sync?	Cari kolom dirty / updated_at, bukan CRUD polos	Ya, mendukung penuh (Lolos). Skema yang dirancang menyertakan kolom <code>dirty INTEGER NOT NULL DEFAULT 0</code> untuk melacak antrean catatan yang belum terunggah, serta <code>updated_at TEXT NOT NULL</code> untuk aturan resolusi konflik <code>last-write-wins</code> . Bahkan dilengkapi indeks khusus (<code>idx_notes_dirty</code> dan <code>idx_notes_updated_at</code>) untuk efisiensi filtering antrean.
Klaim "real-time" AI didukung stream?	Drift watch() / Hive listenable, atau hanya asumsi?	Ya, berbasis API riil (Lolos). AI membedakan secara tegas mekanisme reaktivitas: <code>Drift</code> memiliki native reactive query lewat method <code>watch()</code> (menghasilkan live <code>Stream</code>), <code>Hive</code> menyediakan <code>box.watch()</code> / <code>listenable()</code> , sedangkan pada <code>sqflite</code> secara jujur dijelaskan tidak reaktif bawaan sehingga harus dipicu manual (seperti <code>ref.invalidate()</code> pada Riverpod).
Estimasi boilerplate masuk akal?	Coba flutter pub add + migrasi skema sendiri	Masuk akal & realistis (Lolos). Evaluasi boilerplate sesuai kenyataan teknis: <code>SharedPreferences</code> paling instan tanpa migrasi; <code>sqflite</code> membutuhkan penulisan DDL string manual, mapper <code>toMap/fromMap</code> , dan logika <code>onUpgrade</code> manual; sementara <code>Drift</code> berat di setup awal (<code>build_runner</code> dan code generation), namun sangat minim boilerplate pada operasi query harian.

Langkah 3 Tabel perbandingan storage: docs/perbandingan-storage.md

Draf di bawah berisi analisis awal. **Verifikasi sendiri** (misalnya status paket Hive di pub.dev dan coba pasang Drift) sebelum dikumpulkan.

Kriteria	SharedPrefs	Hive	sqflite	Drift
Kompleksitas query	Tidak ada, hanya get/set per key	Sederhana, filter manual di Dart	Penuh SQL (WHERE, ORDER, JOIN)	Penuh SQL lewat API Dart
Dukungan relasi	Tidak	Tidak native	Ya (FK, JOIN)	Ya
Reaktivitas (stream)	Tidak	Ya (listenable box)	Tidak bawaan, perlu invalidate manual	Ya, <code>watch()</code> berbasis Stream
Type-safety	Rendah (key string)	Sedang (adapter)	Rendah (Map)	Tinggi (code generation)
Ukuran boilerplate	Sangat sedikit	Sedang	Sedang (toMap/fromMap, SQL string)	Banyak (build_runner)
Kemudahan testing	Mudah (mock initial values)	Sedang	Sulit di flutter test, pakai fake	Mudah (in-memory DB)
Cocok untuk preferensi?	Ya	Bisa, berlebihan	Berlebihan	Berlebihan
Cocok untuk 1000+ catatan?	Tidak	Bisa, query terbatas	Ya	Ya
Keputusan & alasan	Dipilih untuk tema & waktu terakhir dibuka	Tidak dipilih: query & relasi terbatas	Dipilih untuk catatan: query, dirty flag, transaksi	Tidak dipilih: boilerplate besar; layak jika butuh stream

Skema untuk 1000+ catatan

Skema `notes` sudah dibuat di Praktikum 2. Untuk data besar, tambahkan indeks agarurut dan hitung dirty tetap cepat (opsional, perlu menaikkan `version` database):

SQL (opsional) [TERMINAL]

```
CREATE INDEX idx_notes_updated ON notes(updated_at DESC);  
CREATE INDEX idx_notes_dirty ON notes(dirty);
```

Langkah 4 Keputusan final

SharedPreferences + SQLite (sqflite). SharedPreferences cukup untuk nilai kecil (tema dan waktu terakhir dibuka) tanpa boilerplate. Catatan adalah koleksi yang butuh urut berdasarkan `updated_at`, update parsial, hitung dirty, dan transaksi, sehingga masuk SQLite. Drift ditolak karena boilerplate code generation tidak sepadan untuk satu tabel; ia layak dipertimbangkan jika aplikasi butuh stream reaktif dan relasi kompleks. Hive ditolak karena query dan relasi terbatas untuk kebutuhan antrean sinkronisasi.

Refactoring Challenge

Tiga butir, tiga commit terpisah. Kerjakan berurutan: butir 1 → 2 → 3.

Butir 1 — Ekstrak widget NoteTile

Hasil: ListTile di NotesPage pindah ke `NoteTile` dan menampilkan chip "Belum tersinkron" bila `note.dirty == true`.

Langkah 1.1 Buat NoteTile

```
lib/widgets/note_tile.dart [BUAT FILE BARU]

import 'package:flutter/material.dart';

import '../data/local/note.dart';

class NoteTile extends StatelessWidget {
  const NoteTile({
    super.key,
    required this.note,
    required this.onTap,
    required this.onDelete,
  });

  final Note note;
  final VoidCallback onTap;
  final VoidCallback onDelete;

  @override
  Widget build(BuildContext context) {
    return ListTile(
      isThreeLine: note.dirty,
      leading: Icon(
        note.dirty ? Icons.cloud_off : Icons.cloud_done,
        color: note.dirty ? Colors.orange : Colors.green,
      ),
      title: Text(note.title),
      subtitle: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Text(
            note.body.isEmpty ? '(tanpa isi)' : note.body,
            maxLines: 1,
            overflow: TextOverflow.ellipsis,
          ),
          if (note.dirty)
            const Padding(
              padding: EdgeInsets.only(top: 4),
              child: Chip(
                label: Text('Belum tersinkron'),
                labelStyle: TextStyle(fontSize: 11),
                visualDensity: VisualDensity.compact,
                materialTapTargetSize: MaterialTapTargetSize.shrinkWrap,
              ),
            ),
        ],
      ),
      onTap: onTap,
      trailing: IconButton(
        tooltip: 'Hapus',
        icon: const Icon(Icons.delete_outline),
        onPressed: onDelete,
      ),
    );
  }
}
```

Langkah 1.2 Pakai NoteTile di NotesPage

Isi file diganti versi lengkap. Bedanya dari Praktikum 4: import `note_tile.dart` ditambah, dan blok `ListTile(...)` diganti `NoteTile(...)`.

```
lib/pages/notes_page.dart    [GANTI SELURUH ISI]

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/local/note.dart';
import '../data/sync.dart';
import '../providers/note_providers.dart';
import '../widgets/note_tile.dart';
import '../widgets/note_form_dialog.dart';
import 'posts_page.dart';
import 'settings_page.dart';

class NotesPage extends ConsumerWidget {
  const NotesPage({super.key});

  Future<void> _openForm(
    BuildContext context,
    WidgetRef ref, [
    Note? note,
  ]) async {
    final result = await showDialog<NoteFormResult>(
      context: context,
      builder: (_) => NoteFormDialog(initial: note),
    );
    if (result == null) return; // dibatalkan
    final actions = ref.read(noteActionsProvider);
    if (note == null) {
      await actions.add(result.title, result.body);
    } else {
      await actions.update(
        note.copyWith(title: result.title, body: result.body),
      );
    }
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final notesAsync = ref.watch(notesProvider);
    final dirty = ref.watch(dirtyCountProvider).value ?? 0;

    return Scaffold(
      appBar: AppBar(
        title: const Text('Offline Notes'),
        actions: [
          Tooltip(
            message: 'Catatan belum tersinkron',
            child: Badge(
              isLabelVisible: dirty > 0,
              label: Text('$dirty'),
              child: const Icon(Icons.cloud_upload_outlined),
            ),
          ),
          IconButton(
            tooltip: 'Posts (cache-first)',
            icon: const Icon(Icons.article_outlined),
            onPressed: () => Navigator.of(context).push(
              MaterialPageRoute(builder: (_) => const PostsPage()),
            ),
          ),
          IconButton(
            tooltip: 'Sinkronkan',
            icon: const Icon(Icons.sync),
            onPressed: () async {
              final messenger = ScaffoldMessenger.of(context);
              try {
                final count = await ref.read(noteActionsProvider).sync();
                messenger.showSnackBar(SnackBar(
                  content: Text(count == 0

```

```

        ? 'Semua catatan sudah tersinkron'
        : '$count catatan berhasil disinkronkan'),
    ));
  } on OfflineException catch (e) {
    messenger.showSnackBar(SnackBar(content: Text(e.message)));
  }
},
),
IconButton(
  tooltip: 'Pengaturan',
  icon: const Icon(Icons.settings_outlined),
  onPressed: () => Navigator.of(context).push(
    MaterialPageRoute(builder: (_) => const SettingsPage()),
  ),
),
],
),
body: notesAsync.when(
  loading: () => const Center(child: CircularProgressIndicator()),
  error: (e, _) => _ErrorView(
    message: 'Gagal membaca database: $e',
    onRetry: () => ref.invalidate(notesProvider),
  ),
  data: (notes) {
    if (notes.isEmpty) return const _EmptyView();
    return ListView.separated(
      itemCount: notes.length,
      separatorBuilder: (context, index) => const Divider(height: 1),
      itemBuilder: (context, index) {
        final note = notes[index];
        return NoteTile(
          note: note,
          onTap: () => _openForm(context, ref, note),
          onDelete: () => ref.read(noteActionsProvider).delete(note.id!),
        );
      },
    );
  },
),
floatingActionButton: FloatingActionButton.extended(
  onPressed: () => _openForm(context, ref),
  icon: const Icon(Icons.add),
  label: const Text('Catatan'),
),
),
}
}
}

```

```

class _EmptyView extends StatelessWidget {
  const _EmptyView();

  @override
  Widget build(BuildContext context) {
    return const Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Icon(Icons.note_alt_outlined, size: 64),
          SizedBox(height: 12),
          Text('Belum ada catatan. Tekan + untuk menambah.'),
        ],
      ),
    );
  }
}

```

```

class _ErrorView extends StatelessWidget {
  const _ErrorView({required this.message, required this.onRetry});

  final String message;
  final VoidCallback onRetry;

  @override
  Widget build(BuildContext context) {

```

```

return Center(
  child: Padding(
    padding: const EdgeInsets.all(24),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        const Icon(Icons.error_outline, size: 48),
        const SizedBox(height: 12),
        Text(message, textAlign: TextAlign.center),
        const SizedBox(height: 12),
        FilledButton(onPressed: onRetry, child: const Text('Coba lagi')),
      ],
    ),
  ),
);
}

```

Langkah 1.3 Uji

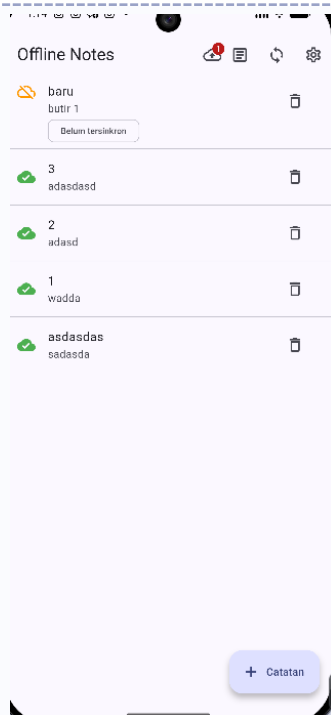
Terminal [TERMINAL]

```

flutter analyze
flutter run

```

No	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
1	Chip muncul	Tambah catatan baru	Catatan menampilkan chip Belum tersinkron	Catatan menampilkan chip Belum tersinkron	☐ Lulus
2	Chip hilang	Tekan Sinkronkan (online)	Chip hilang, ikon awan hijau	Chip hilang, ikon awan hijau	☐ Lulus
3	Tap & hapus	Ketuk catatan, lalu hapus catatan lain	Dialog ubah terbuka; hapus tetap berfungsi	Dialog ubah terbuka; hapus tetap berfungsi	☐ Lulus



Gambar 15. NoteTile dengan chip "Belum tersinkron"

simpan: screenshots/r1-notetile-chip.png

Commit praktikum ini:

Terminal [TERMINAL]

```
git add .
git commit -m "refactor(widgets): ekstrak NoteTile dengan chip belum tersinkron"
```

Butir 2 — Pisahkan cache posts dari logika sync

Audit kondisi saat ini terhadap aturan modul:

Berkas	Seharusnya hanya berisi	Kondisi	Tindakan
<code>note_repository.dart</code>	CRUD catatan	Sudah benar (<code>countDirty</code> & <code>markAllSynced</code> hanya akses data catatan)	Tidak diubah
<code>post_repository.dart</code>	Cache posts	Sudah benar	Tidak diubah
<code>sync.dart</code>	<code>syncNotes</code> + aturan konflik	Masih memuat <code>OfflineException</code> yang juga dipakai posts	Pindahkan ke berkas sendiri

Langkah 2.1 Buat `offline_exception.dart`

`lib/data/offline_exception.dart` [BUAT FILE BARU]

```
class OfflineException implements Exception {
  const OfflineException(this.message);

  final String message;

  @override
  String toString() => 'OfflineException: $message';
}
```

Langkah 2.2 Rapikan `sync.dart`

`lib/data/sync.dart` [GANTI SELURUH ISI]

```
import 'local/note.dart';
import 'offline_exception.dart';
import 'repositories/note_repository.dart';

/// Mengirim catatan dirty ke "server" (simulasi) lalu menandainya bersih.
/// Mengembalikan jumlah catatan yang tersinkron.
Future<int> syncNotes(
  NoteRepository repo, {
    bool offline = false,
    Duration latency = const Duration(seconds: 1),
  }) async {
  if (offline) {
    throw const OfflineException('Perangkat offline, sinkronisasi ditunda.');
```

```
  }
  final dirtyCount = await repo.countDirty();
  if (dirtyCount == 0) return 0;

  // Simulasi upload. Pada project nyata: kirim tiap catatan dirty
  // ke REST API, lalu tandai bersih HANYA bila server menjawab 2xx.
  await Future.delayed(latency);
  await repo.markAllSynced();
  return dirtyCount;
}

/// Aturan konflik: last-write-wins berdasarkan updatedAt.
Note resolveConflict(Note local, Note remote) {
  return remote.updatedAt.isAfter(local.updatedAt) ? remote : local;
```

```
}
```

Langkah 2.3 Perbarui import yang memakai OfflineException

Ada tiga berkas lain yang memakai `OfflineException`. Ubah satu baris import di masing-masing:

lib/providers/post_providers.dart [UBAH]

```
// GANTI baris ini:  
import '../data/sync.dart';  
// MENJADI:  
import '../data/offline_exception.dart';
```

lib/pages/notes_page.dart [UBAH]

```
// GANTI baris ini:  
import '../data/sync.dart';  
// MENJADI:  
import '../data/offline_exception.dart';
```

test/note_test.dart [UBAH]

```
// TAMBAHKAN satu baris ini (di bawah import note.dart), sync.dart tetap dipertahankan:  
import 'package:week5_offline_notes/data/offline_exception.dart';
```

Di `lib/providers/note_providers.dart` **tidak ada perubahan**: ia hanya memanggil `syncNotes`, jadi tetap mengimpor `sync.dart`.

Langkah 2.4 Verifikasi

Terminal [TERMINAL]

```
flutter analyze  
flutter test
```

Cek pemisahan tanggung jawab (hasil yang benar: **tidak ada output**):

Git Bash / macOS / Linux [TERMINAL]

```
grep -nE "cached_posts|dio|OfflineException" lib/data/repositories/note_repository.dart  
grep -nE "class |cached_posts|dio" lib/data/sync.dart
```

PowerShell (Windows) [TERMINAL]

```
Select-String -Path lib/data/repositories/note_repository.dart -Pattern  
"cached_posts|dio|OfflineException"  
Select-String -Path lib/data/sync.dart -Pattern "class |cached_posts|dio"
```

Harus terjadi

`flutter analyze` bersih dan 7 test masih lulus. Aplikasi berperilaku sama seperti sebelumnya (refactor murni, tanpa perubahan fitur).

```

● PS C:\laragon\www\244107020136-mobile-course\07
-week-7\week5_offline_notes> flutter analyze
Analyzing week5_offline_notes...

No issues found! (ran in 4.3s)
PS C:\laragon\www\244107020136-mobile-course\07
● -week-7\week5_offline_notes> flutter test
00:04 +7: All tests passed!
○ PS C:\laragon\www\244107020136-mobile-course\07
-week-7\week5_offline_notes>

```

Gambar 16. flutter analyze & flutter test setelah refactor
simpan: screenshots/r2-analyze-test.png

Commit praktikum ini:

Terminal [TERMINAL]

```
git add .
git commit -m "refactor(data): pisahkan OfflineException dari sync dan rapikan tanggung jawab repository"
```

Butir 3 — Halaman detail dengan GoRouter

Hasil: rute `/note/:id` membaca catatan langsung dari repository lokal lewat `noteByIdProvider`, bukan dari state halaman list.

Langkah 3.1 Pasang go_router

Terminal [TERMINAL]

```
flutter pub add go_router
```

Langkah 3.2 Tambah noteByIdProvider

Buka `lib/providers/note_providers.dart`, lakukan dua perubahan:

`lib/providers/note_providers.dart` (perubahan 1) [UBAH]

```
// TAMBAHKAN di bawah dirtyCountProvider:

final noteByIdProvider = FutureProvider.family<Note?, int>(
  (ref, id) => ref.watch(noteRepositoryProvider).getNoteById(id),
);
```

`lib/providers/note_providers.dart` (perubahan 2) [UBAH]

```
// Di dalam method _refresh(), TAMBAHKAN satu baris:
void _refresh() {
  _ref.invalidate(notesProvider);
  _ref.invalidate(dirtyCountProvider);
  _ref.invalidate(noteByIdProvider); // <- baris baru
}
```

Baris invalidate ini membuat halaman detail ikut memuat ulang setelah catatan diubah.

Langkah 3.3 Halaman detail

lib/pages/note_detail_page.dart [BUAT FILE BARU]

```
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import '../data/local/note.dart';
import '../providers/note_providers.dart';
import '../widgets/note_form_dialog.dart';

class NoteDetailPage extends ConsumerWidget {
  const NoteDetailPage({super.key, required this.id});

  final int id;

  Future<void> _edit(WidgetRef ref, BuildContext context, Note note) async {
    final result = await showDialog<NoteFormResult>(
      context: context,
      builder: (_) => NoteFormDialog(initial: note),
    );
    if (result == null) return;
    await ref.read(noteActionsProvider).update(
      note.copyWith(title: result.title, body: result.body),
    );
  }

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    // Dibaca langsung dari repository lokal, bukan dari state halaman list.
    final noteAsync = ref.watch(noteByIdProvider(id));
    final note = noteAsync.value;

    return Scaffold(
      appBar: AppBar(title: const Text('Detail catatan')),
      body: noteAsync.when(
        loading: () => const Center(child: CircularProgressIndicator()),
        error: (e, _) => Center(child: Text('Gagal membaca catatan: $e')),
        data: (current) {
          if (current == null) {
            return const Center(child: Text('Catatan tidak ditemukan'));
          }
          return ListView(
            padding: const EdgeInsets.all(16),
            children: [
              Text(
                current.title,
                style: Theme.of(context).textTheme.headlineSmall,
              ),
              const SizedBox(height: 8),
              Wrap(
                spacing: 8,
                children: [
                  Chip(
                    avatar: Icon(
                      current.dirty ? Icons.cloud_off : Icons.cloud_done,
                      size: 18,
                      color: current.dirty ? Colors.orange : Colors.green,
                    ),
                    label: Text(
                      current.dirty ? 'Belum tersinkron' : 'Sudah tersinkron',
                    ),
                  ),
                  Chip(
                    label: Text(
                      'Diubah ${current.updatedAt.toLocal().toString().split('.').first}',
                    ),
                  ),
                ],
              ),
              const Divider(height: 32),
              Text(
                current.body.isEmpty ? '(tanpa isi)' : current.body,
                style: Theme.of(context).textTheme.bodyLarge,
              ),
            ],
          );
        },
      ),
    );
  }
}
```

```

    ),
  ],
);
},
),
floatingActionButton: note == null
? null
: FloatingActionButton(
  tooltip: 'Ubah catatan',
  onPressed: () => _edit(ref, context, note),
  child: const Icon(Icons.edit_outlined),
),
);
}
}
}

```

Langkah 3.4 Konfigurasi router

lib/router.dart [BUAT FILE BARU]

```

import 'package:go_router/go_router.dart';

import 'pages/note_detail_page.dart';
import 'pages/notes_page.dart';

final router = GoRouter(
  routes: [
    GoRoute(
      path: '/',
      builder: (context, state) => const NotesPage(),
      routes: [
        GoRoute(
          path: 'note/:id',
          builder: (context, state) => NoteDetailPage(
            id: int.parse(state.pathParameters['id']!),
          ),
        ),
      ],
    ),
  ],
);

```

Langkah 3.5 Ganti MaterialApp dengan MaterialApp.router

lib/main.dart [GANTI SELURUH ISI]

```

import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

import 'providers/prefs_providers.dart';
import 'router.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  runApp(const ProviderScope(child: OfflineNotesApp()));
}

class OfflineNotesApp extends ConsumerWidget {
  const OfflineNotesApp({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final isDark = ref.watch(darkModeProvider).value ?? false;

    return MaterialApp.router(
      title: 'Offline Notes',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(

```

```

        colorSchemeSeed: Colors.indigo,
        brightness: Brightness.light,
    ),
    darkTheme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        brightness: Brightness.dark,
    ),
    themeMode: isDark ? ThemeMode.dark : ThemeMode.light,
    routerConfig: router,
);
}
}

```

Langkah 3.6 Navigasi dari daftar catatan

Di `lib/pages/notes_page.dart`, lakukan **dua perubahan**:

lib/pages/notes_page.dart (perubahan 1: import) [UBAH]

```

// TAMBAHKAN di bagian import:
import 'package:go_router/go_router.dart';

```

lib/pages/notes_page.dart (perubahan 2: onTap) [UBAH]

```

// GANTI baris ini (di dalam NoteTile):
onTap: () => _openForm(context, ref, note),
// MENJADI:
onTap: () => context.push('/note/${note.id}'),

```

Perhatian

Setelah perubahan ini mengetuk catatan membuka **halaman detail**. Fungsi ubah catatan pindah ke tombol pensil di halaman detail, sedangkan tombol **+ Catatan** tetap memakai `_openForm`.

Langkah 3.7 Jalankan dan uji

Karena ada paket baru dan perubahan struktur, stop aplikasi lalu jalankan ulang.

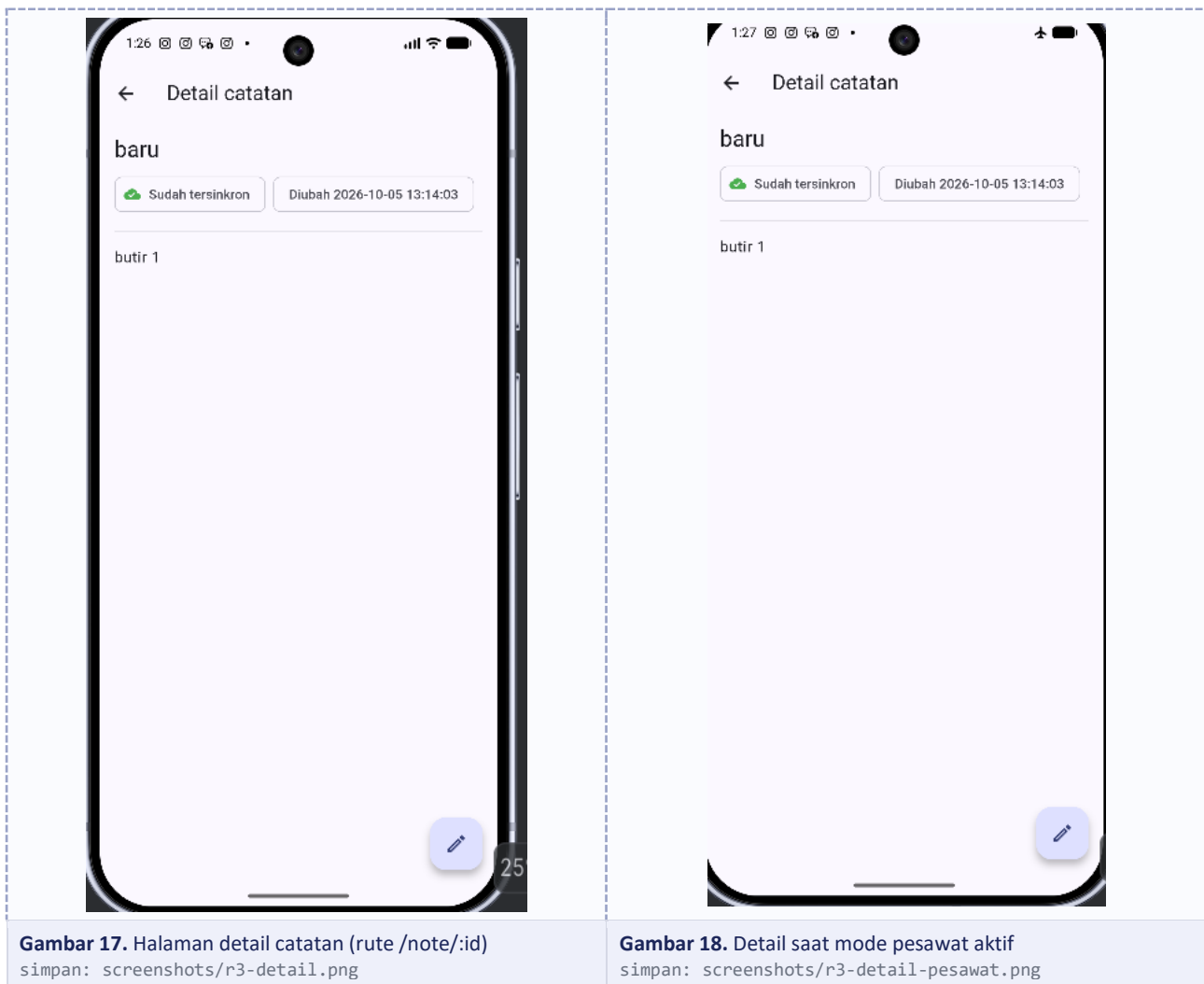
Terminal [TERMINAL]

```

flutter analyze
flutter run

```

No	Skenario	Langkah	Hasil diharapkan	Hasil aktual	Status
1	Buka detail	Ketuk satu catatan di daftar	Halaman detail tampil: judul, isi, chip sinkron, waktu ubah		☐ Lulus ☐ Gagal
2	Kembali	Tekan tombol back	Kembali ke daftar		☐ Lulus ☐ Gagal
3	Ubah dari detail	Tekan tombol pensil, ubah isi, Simpan	Detail langsung berubah; di daftar catatan naik ke atas dan chip dirty muncul		☐ Lulus ☐ Gagal
4	Mode pesawat	Aktifkan mode pesawat, buka detail catatan	Tetap tampil (data dari SQLite lokal)		☐ Lulus ☐ Gagal
5	Tema	Ganti tema di Pengaturan, kembali ke detail	Tema konsisten (MaterialApp.router tetap memakai darkModeProvider)		☐ Lulus ☐ Gagal



Alasan teknis (untuk demo)

R-GR Mengapa detail membaca dari repository, bukan dari state halaman list?

Rute bisa dibuka langsung (deep link) atau setelah proses direstart, saat state halaman list belum ada. Dengan `noteByIdProvider` data selalu diambil dari sumber kebenaran (SQLite), sehingga detail benar di kondisi apa pun dan ikut diperbarui lewat `invalidate`.

Commit praktikum ini:

Terminal [TERMINAL]

```
git add .
git commit -m "feat(router): halaman detail catatan dengan GoRouter"
```

Refleksi dan Kesimpulan

Pertanyaan refleksi

R1 Mengapa daftar catatan tidak boleh disimpan di SharedPreferences? Apa yang rusak?

Daftar harus disimpan sebagai satu string JSON, jadi tiap perubahan satu catatan berarti membaca, mengurai, mengubah, dan menulis ulang seluruh daftar. Tidak ada query,urut, atau update parsial; dirty flag per catatan sulit dikelola; performa turun saat catatan banyak; dan risiko data rusak/tertipa lebih besar.

R2 Kapan cache-first cukup, kapan butuh strategi lain?

Cache-first cukup untuk data yang boleh agak lama seperti artikel, katalog, atau posts. Untuk data yang basi-nya berbahaya (harga real-time, saldo, stok) dipakai network-first agar yang tampil selalu data terbaru, dengan cache hanya sebagai cadangan.

R3 Bagaimana dirty flag menjadi antrean sync tanpa memblokir UI? Kapan perlu tabel outbox?

Setiap perubahan langsung ditulis ke SQLite dengan `dirty = 1`, jadi UI tidak menunggu jaringan. Sinkronisasi berjalan asinkron terpisah dan hanya memproses baris dirty. Tabel outbox diperlukan ketika harus merekam operasi (terutama delete dan urutan operasi) atau butuh retry per operasi, bukan hanya status akhir catatan.

R4 Bagian mana dari rekomendasi AI yang kamu tolak, dan mengapa?

(Isi dari hasil AI-mu sendiri.) Contoh format jawaban: "AI menyarankan [..]. Saya tolak karena [..] setelah saya verifikasi dengan [..]."

MINI ASSIGMENT ADA PADA GITHUB BESERTA DOKUMENTASINYA

Kesimpulan

Hal terpenting yang saya pelajari adalah bahwa aplikasi offline-first menjadikan database lokal sebagai sumber data utama: UI membaca dan menulis ke SQLite lewat repository dan provider, sehingga tetap berfungsi tanpa internet. Perubahan yang belum terkirim ditandai dengan dirty flag lalu disinkronkan belakangan dengan aturan konflik yang jelas (last-write-wins). Memisahkan UI, provider, dan repository juga membuat kode mudah diuji memakai repository palsu tanpa menyentuh database sungguhan.